

Mike Ichikawa

DATA SCIENTIST · ML ENGINEER · DATA ENGINEER

Portland, OR projects.ichikawa@gmail.com mtichikawa.github.io github.com/mtichikawa

SUMMARY

MS Mathematics (CCNY, 2014) and BS Mechanical Engineering (UC Berkeley). Three years in semiconductor Physical Design at Intel; nine years teaching undergraduate mathematics at Santa Rosa Junior College. Over the past eighteen months, a self-directed transition into data science and ML engineering — culminating in Microclaw, a production AI agent for Microsoft 365 (KNN tool selection, function-calling across 12 services) that I built solo, launching on the Microsoft Commercial Marketplace. Complementary portfolio covers time series forecasting, anomaly detection, NLP, cloud ETL, ML serving, a Databricks medallion lakehouse, C++ performance extensions via pybind11, and a five-repo paper trading system.

TECHNICAL SKILLS

Languages	Python · SQL · C++ · R · TypeScript · Node.js
ML / Statistics	scikit-learn · Prophet · PyTorch · XGBoost · LightGBM · scipy · statsmodels · Bayesian inference
LLM / NLP	Anthropic API · Azure OpenAI · Microsoft Graph · MSAL · HuggingFace · FinBERT · prompt engineering
Data Engineering	PostgreSQL · Databricks · Delta Lake · medallion · dbt · SQLAlchemy · pandas
Cloud / Infra	AWS (S3 · Lambda · DynamoDB · CloudWatch) · Docker · FastAPI · Redis · Pydantic v2 · boto3 · Microsoft Commercial Marketplace
Tools	Git · pytest · Vitest · Jupyter · Streamlit · Docker Compose

EXPERIENCE

Founder & Engineer

2026 – present

Microclaw LLC · Portland, OR

- Built a production AI agent on Microsoft Teams (Azure OpenAI GPT-4o + function-calling) operating across 12 Microsoft 365 services through 54 tool functions on Microsoft Graph; launching on Microsoft AppSource
- Implemented KNN tool selection over 1,020 synthetic training examples (k=7, 0.80 confidence threshold, cosine-similarity fallback): 95.0% recall vs 92.7% embedding-only baseline, 6.4 tools selected per request vs 18, ~60% input-token reduction
- Designed semantic memory retrieval (cosine similarity over per-user persistent context) and a smart model router (GPT-4o-mini default, escalation to GPT-4o on complexity signals); built an in-house eval framework scoring agent output across 75 tasks against a baseline bot
- Built a natural-language automations layer where users describe IFTTT-style rules in plain English; the agent parses intent into structured rules persisted to SQLite, fired in real-time via Microsoft Graph change-notification webhooks

Mathematics Faculty

2015 – 2023

Santa Rosa Junior College · Santa Rosa, CA

- Taught undergraduate mathematics for nine years: Algebra through Calculus III, Linear Algebra, Differential Equations, Statistics
- Designed assessment systems and drove data-driven iterative improvements to course materials based on longitudinal performance analysis across multiple cohorts
- Routinely explained complex mathematical concepts clearly to students with varying preparation levels

Physical Design Engineer

2008 – 2011

Intel Corporation · Semiconductor R&D

- Microprocessor physical design on Intel Ivy Bridge (22nm) — timing closure, power analysis, design rule verification
- Built Python and Perl automation for physical verification workflows, processing millions of layout data points per run

EDUCATION

MS Mathematics	The City College of New York	2014
BS Mechanical Engineering	University of California, Berkeley	2008

PORTFOLIO PROJECTS — full portfolio at mtichikawa.github.io

Trading System Arc

[Live Demo](#) · github.com/mtichikawa — 5 repos

Five interconnected repositories: live OHLCV pipeline (ccxt/Kraken + PostgreSQL) → candlestick chart generation (mplfinance) → dual-path signal engine (EMA/RSI/MACD/BB + local FinBERT sentiment, 0.6/0.4 fusion) → parameter-optimizing backtester (Sharpe/Sortino/drawdown, staged grid search, T3 feedback loop) → Streamlit multi-page oversight dashboard with live signal display, equity curve, and parameter review. 174/174 tests across T2–T5.

[ccxt](#) · [PostgreSQL](#) · [mplfinance](#) · [FinBERT](#) · [pandas](#) · [NumPy](#) · [Streamlit](#) · [Plotly](#) · [pytest](#)

Databricks Medallion Lakehouse

github.com/mtichikawa/databricks-lakehouse

Medallion lakehouse (bronze → silver → gold) on 10M NYC taxi rows with Delta Lake. Append-only bronze with provenance metadata; silver types, dedupes, and quarantines invalid rows rather than dropping them; gold serves query-specific aggregates. Row-level data quality gates between layers halt the pipeline on schema drift or null-rate breaches. 95 tests across bronze/silver/gold/quality/pipeline. Runs locally via deltalake; deployable to Databricks Community Edition.

[Delta Lake](#) · [medallion architecture](#) · [pandas](#) · [pyarrow](#) · [data quality gates](#) · [pytest](#)

Real-Time Anomaly Detection

github.com/mtichikawa/anomaly-detection

Streaming ensemble detector: Isolation Forest, LSTM autoencoder, and Z-score with 2-of-3 majority voting. Dockerized FastAPI REST API with health checks and configurable detector selection. Live interactive demo on portfolio site.

[Docker](#) · [FastAPI](#) · [scikit-learn](#) · [PyTorch](#) · [Redis](#) · [pytest](#)

Cloud ETL Pipeline

github.com/mtichikawa/cloud-etl-pipeline

AWS-native ETL: tenacity-based REST API ingestion with exponential backoff → Parquet/Snappy storage on S3 with Hive date partitioning → Lambda validation → DynamoDB run logging for failure traceability. Tests mock all AWS calls with moto.

[AWS S3](#) · [Lambda](#) · [DynamoDB](#) · [boto3](#) · [pandas](#) · [Apache Parquet](#) · [moto](#)

SQL Analytics Pipeline

github.com/mtichikawa/sql-analytics-pipeline

Three-layer dbt-style pipeline (staging → clean → marts) on NYC Airbnb data. Window functions, CTEs, SQLAlchemy ORM with swappable SQLite/PostgreSQL backend. Layered transform pattern enforces raw data immutability.

[PostgreSQL](#) · [SQLAlchemy](#) · [dbt-style transforms](#) · [SQLite](#) · [pandas](#)

LLM Data Analysis Assistant

github.com/mtichikawa/llm-data-assistant

Two-path query routing: rule-based fast path handles ~70% of queries (correlations, aggregations, group-bys) with zero API cost. Queries below confidence threshold escalate to Anthropic API with DataFrame context injection and multi-turn conversation history.

[Anthropic API](#) · [pandas](#) · [hybrid routing](#) · [multi-turn conversation](#)

Additional: Multi-Armed Bandit A/B Testing (Thompson Sampling · UCB1 · Bayesian inference) · Bias Detection in LLMs (ANOVA · Cohen's d) · Financial NLP Parser (SEC EDGAR · regex · sentiment lexicon) · GitHub Trend Forecaster (Prophet · PyTorch LSTM in progress)